Practical : 5

AIM:

To perform CRUD operations and aggregation operations in MongoDB using MongoDB Compass and MongoDB Shell.

PROBLEM DEFINITION:

Scenario: Perform Create, Read, Update, and Delete (CRUD) operations using MongoDB. Students will work with a NoSQL document database to understand how unstructured and semi-structured data is managed at scale in Big Data environments.

Tasks:

1. Install and configure MongoDB.
2. Create a database and collection.
3. Insert documents (single and multiple).
4. Read/Query documents using various filters.
5. Update existing documents.
6. Delete documents and collections.
7. Use MongoDB aggregation pipeline for data analysis.

OUTPUT :

Task 1 — Start MongoDB

```
(c) Microsoft Corporation. All rights reserved.

C:\Windows\System32>net start MongoDB
The requested service has already been started.

More help is available by typing NET HELPMSG 2182.


C:\Windows\System32>sc query MongoDB

SERVICE_NAME: MongoDB
        TYPE               : 10  WIN32_OWN_PROCESS
        STATE              : 4  RUNNING
                                (STOPPABLE, NOT_PAUSABLE, ACCEPTS_PRESHUTDOWN)
        WIN32_EXIT_CODE    : 0  (0x0)
        SERVICE_EXIT_CODE  : 0  (0x0)
        CHECKPOINT         : 0x0
        WAIT_HINT          : 0x0
```

```
C:\Windows\System32>mongosh
Current Mongosh Log ID: 6aaac4b85784c819a71e2620
Connecting to:          mongodb://127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=2000&a
ppName=mongosh+2.5.10
Using MongoDB:          8.2.0
Using Mongosh:          2.5.10
mongosh 2.8.2 is available for download: https://www.mongodb.com/try/download/shell

For mongosh info see: https://www.mongodb.com/docs/mongodb-shell/

------
   The server generated these startup warnings when booting
   2026-09-16T18:06:04.424+05:30: Access control is not enabled for the database. Read and write access
to data and configuration is unrestricted
------

test>
```

Task 2 — Create Database and Collection

```
test> use college
switched to db college
```

```
test> use college
switched to db college
college> db.createCollection("students")
{ ok: 1 }
college> show collections
students
college>
```

Task 3 — Insert Documents

```
college> db.students.insertOne({
...     roll_no: 101,
...     name: "Rahul",
...     age: 20,
...     branch: "CSE",
...     marks: 85
... })
{
  acknowledged: true,
  insertedId: ObjectId('6aaac5ee628ebe3e291e2621')
}
college>
```

```
college> db.students.insertMany([
...     {roll_no: 102, name: "Priya", age: 21, branch: "IT", marks: 92},
...     {roll_no: 103, name: "Amit", age: 20, branch: "CSE", marks: 76},
...     {roll_no: 104, name: "Neha", age: 22, branch: "ECE", marks: 88},
...     {roll_no: 105, name: "Karan", age: 21, branch: "IT", marks: 69}
... ])
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('6aaac607628ebe3e291e2622'),
    '1': ObjectId('6aaac607628ebe3e291e2623'),
    '2': ObjectId('6aaac607628ebe3e291e2624'),
    '3': ObjectId('6aaac607628ebe3e291e2625')
  }
}
college>
```

Task 4 — Read / Query Documents

First display everything

```
college> db.students.find()
[
  {
    _id: ObjectId('6aaac5ee628ebe3e291e2621'),
    roll_no: 101,
    name: 'Rahul',
    age: 20,
    branch: 'CSE',
    marks: 85
  },
  {
    _id: ObjectId('6aaac607628ebe3e291e2622'),
    roll_no: 102,
    name: 'Priya',
    age: 21,
    branch: 'IT',
    marks: 92
  },
  {
    _id: ObjectId('6aaac607628ebe3e291e2623'),
    roll_no: 103,
    name: 'Amit',
    age: 20,
    branch: 'CSE',
    marks: 76
  },
```

Find one student

```
college> db.students.findOne({roll_no: 101})
{
  _id: ObjectId('6aaac5ee628ebe3e291e2621'),
  roll_no: 101,
  name: 'Rahul',
  age: 20,
  branch: 'CSE',
  marks: 85
}
college>
```

Students having marks greater than 80

```
college> db.students.find({marks: {$gt: 80}}).pretty()
[
  {
    _id: ObjectId('6aaac5ee628ebe3e291e2621'),
    roll_no: 101,
    name: 'Rahul',
    age: 20,
    branch: 'CSE',
    marks: 85
  },
  {
    _id: ObjectId('6aaac607628ebe3e291e2622'),
    roll_no: 102,
    name: 'Priya',
    age: 21,
    branch: 'IT',
    marks: 92
  },
```

Marks between 70 and 90

```
college> db.students.find({
...     marks: {$gte: 70, $lte: 90}
... }).pretty()
[
  {
    _id: ObjectId('6aaac5ee628ebe3e291e2621'),
    roll_no: 101,
    name: 'Rahul',
    age: 20,
    branch: 'CSE',
    marks: 85
  },
  {
    _id: ObjectId('6aaac607628ebe3e291e2623'),
    roll_no: 103,
    name: 'Amit',
    age: 20,
    branch: 'CSE',
    marks: 76
  },
```

Task 5 — Update Documents

5.1 Update one student

```
college> db.students.updateOne(
...     {roll_no: 101},
...     {$set: {marks: 90}}
... )
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
college> db.students.findOne({roll_no: 101})
{
  _id: ObjectId('6aaac5ee628ebe3e291e2621'),
  roll_no: 101,
  name: 'Rahul',
  age: 20,
  branch: 'CSE',
  marks: 90
}
college>
```

Old mark as below

```
_id: ObjectId('6aaac5ee628ebe3e291e2621'),
roll_no: 101,
name: 'Rahul',
age: 20,
branch: 'CSE',
marks: 85
```

5.2 Update multiple documents

```
college> db.students.updateMany(
...     {branch: "CSE"},
...     {$inc: {marks: 5}}
... )
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 2,
  modifiedCount: 2,
  upsertedCount: 0
}
college> db.students.find({branch: "CSE"}).pretty()
[
  {
    _id: ObjectId('6aaac5ee628ebe3e291e2621'),
    roll_no: 101,
    name: 'Rahul',
    age: 20,
    branch: 'CSE',
    marks: 95
  },
  {
    _id: ObjectId('6aaac607628ebe3e291e2623'),
    roll_no: 103,
    name: 'Amit',
    age: 20,
    branch: 'CSE',
    marks: 81
  }
]
college>
```

Task 6 — Delete Documents

Delete student with roll number 105:

```
college> db.students.deleteOne({roll_no: 105})
{ acknowledged: true, deletedCount: 1 }
college> db.students.find().pretty()
[
  {
    _id: ObjectId('6aaac5ee628ebe3e291e2621'),
    roll_no: 101,
    name: 'Rahul',
    age: 20,
    branch: 'CSE',
    marks: 95
  },
  {
    _id: ObjectId('6aaac607628ebe3e291e2622'),
    roll_no: 102,
    name: 'Priya',
    age: 21,
    branch: 'IT',
    marks: 92
  },
  {
    _id: ObjectId('6aaac607628ebe3e291e2623'),
    roll_no: 103,
    name: 'Amit',
    age: 20,
    branch: 'CSE',
    marks: 81
  },
  {
    _id: ObjectId('6aaac607628ebe3e291e2624'),
    roll_no: 104,
    name: 'Neha',
    age: 22,
    branch: 'ECE',
    marks: 88
  }
]
college>
```

6.2 Delete based on condition

```
college> db.students.deleteMany({marks: {$lt: 70}})
{ acknowledged: true, deletedCount: 0 }
college>
```

```
college> db.students.deleteMany({marks: {$lt: 88}})
{ acknowledged: true, deletedCount: 1 }
college>
```

Task 7 — MongoDB Aggregation Pipeline

7.1 Find average marks for each branch

```
college> db.students.aggregate([
...     {
...         $group: {
...             _id: "$branch",
...             averageMarks: { $avg: "$marks" }
...         }
...     }
... ])
[
  { _id: 'ECE', averageMarks: 88 },
  { _id: 'CSE', averageMarks: 95 },
  { _id: 'IT', averageMarks: 92 }
]
college>
```

7.2 Find maximum marks in each branch

```
college> db.students.aggregate([
...     {
...         $group: {
...             _id: "$branch",
...             averageMarks: { $avg: "$marks" }
...         }
...     }
... ])
[
  { _id: 'ECE', averageMarks: 88 },
  { _id: 'CSE', averageMarks: 95 },
  { _id: 'IT', averageMarks: 92 }
]
college> db.students.aggregate([
...     {
...         $group: {
...             _id: "$branch",
...             maximumMarks: { $max: "$marks" }
...         }
...     }
... ])
[
  { _id: 'IT', maximumMarks: 92 },
  { _id: 'CSE', maximumMarks: 95 },
  { _id: 'ECE', maximumMarks: 88 }
]
college>
```

7.3 Count students in each branch

```
college> db.students.aggregate([
...     {
...         $group: {
...             _id: "$branch",
...             totalStudents: { $sum: 1 }
...         }
...     }
... ])
[
  { _id: 'IT', totalStudents: 1 },
  { _id: 'CSE', totalStudents: 1 },
  { _id: 'ECE', totalStudents: 1 }
]
college>
```

7.4 Filter + Aggregation

```
college> db.students.aggregate([
...     {
...         $match: {
...             marks: { $gte: 80 }
...         }
...     },
...     {
...         $group: {
...             _id: "$branch",
...             averageMarks: { $avg: "$marks" }
...         }
...     }
... ])
[
  { _id: 'IT', averageMarks: 92 },
  { _id: 'CSE', averageMarks: 95 },
  { _id: 'ECE', averageMarks: 88 }
]
college>
```

Task 7.5 — Create an Index

```
college> db.students.createIndex({roll_no: 1})
roll_no_1
college> db.students.getIndexes()
[
  { v: 2, key: { _id: 1 }, name: '_id_' },
  { v: 2, key: { roll_no: 1 }, name: 'roll_no_1' }
]
college>
```

KEY QUESTIONS :

Q1. What is MongoDB and how does it differ from relational databases?

MongoDB is a NoSQL document database that stores data in flexible documents instead of tables. Unlike relational databases, MongoDB does not require a fixed schema and is suitable for handling large and changing datasets.

Q2. What are CRUD operations and why are they fundamental to database management?

CRUD stands for Create, Read, Update, and Delete. These operations are fundamental because they allow users to insert, view, modify, and remove data from the database.

Q3. How does MongoDB store data (BSON format)?

MongoDB stores data in BSON (Binary JSON) format. BSON stores data as key-value pairs and supports different data types such as strings, numbers, arrays, dates, and embedded documents.

Q4. What is the aggregation pipeline in MongoDB?

The aggregation pipeline is a sequence of stages used to process and analyze documents. It can filter, group, sort, and calculate data using stages such as $match, $group, $sort, and $project.

Q5. When should NoSQL databases be preferred over SQL databases?

NoSQL databases should be preferred when data is unstructured or changes frequently, and when large amounts of data need to be distributed and scaled easily. They are commonly used in Big Data and real-time applications.

CONCLUSION :

Thus, CRUD operations were successfully performed using MongoDB. Documents were inserted, retrieved, updated and deleted using MongoDB Shell. MongoDB Compass was used to visualize the database and documents. The aggregation pipeline was also used to analyze student data by calculating average, maximum, minimum and count values.